# COMP3011 Computer Graphics

# Spring 2026

# Assessment 1

## Introduction

This coursework will test your understanding of both the 3D computer graphics rendering pipeline and ray tracing, as taught in the course. You will need to program a rasteriser and a raytracer. On Moodle you will find 2 Visual Studio Solutions, which are zipped up and contain everything you will need to complete your coursework. **You cannot use any other API**.

## Submission

You need to submit two separate files to Moodle:

1. 'rasterise.h' for Part A.
2. 'raytrace.h' for Part B.

## Testing

Before submitting your code, test it on an A32 lab PC. The marker will download your submitted file from Moodle, add it to the appropriate Visual Studio .sln file, click the Run Button, and it should compile and run without any further interventions.

# Part A - Rasteriser

In Part A you need to program a software rasteriser. Only edit `'rasterise.h'` and submit only that single file to Moodle.

Open `Assessment1A.cpp` and look in `main()`. After some variables for performance analysis are declared, there is a variable to store the path of the OBJ file to render, which is parsed using `obj_parse()` and the geometry is stored in a variable called `tris`.

Have a look at `obj_parse()` in "`do_not_edit.h`". You will see that `tiny_obj_loader` is loading the vertex data, which is then parsed into lists of positions, normal, and colours. These are used to construct a list of triangles. There is a special rule for "cornell2" to set the colour of triangles. Do not change any of this file.

In `main()`, the list of triangles is passed to the method `render()` and then the `colour_buffer` is saved to a file called "`render.bmp`". Run the program and "`render.bmp`" should be generated, and you can open it in Microsoft paint, and should see that it is an entirely black image.

Your task is to implement `render()` and all the necessary functions to rasterise the triangles as declared in "`rasterise.h`". You need to implement all the methods in "`rasterise.h`".

Make sure

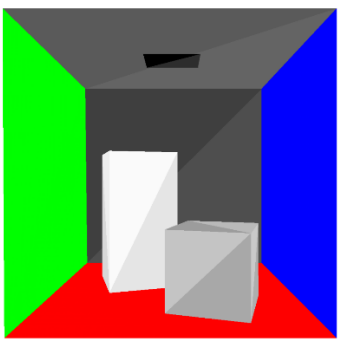
i) everything you implement is in "`rasterise.h`" since that is the only file you can submit,
ii) your rasteriser renders the triangles passed into `render()` from main and
iii) you use the method `writeColToDisplayBuffer()` implemented in "`do_not_edit.h`" to write to the `colour_buffer` which is written to a file at the end of `main()`.

You need to implement from the Render loop in Chapter 3 Graphics Programming, Clear the i) colour buffer and ii) depth buffer. `colour_buffer` and `depth_buffer` can be found at the top of "`do_not_edit.h`".

You need to implement the following items from the rendering pipeline as can be seen in Chapter 6 Spaces and Chapter 7 The Rendering Pipeline.

1. Vertex Transformation. You have been provided with a test OBJ file called "cornell2", which requires translation of t= (0.1, -2.5, -6) to the model using `glm::translate()`. Use the following parameters for the projection matrix `fovy = 55.f`, `zNear =0.1f`, `zFar =10.f` using `glm::perspective()`.
2. Perspective Division.
3. Viewport Transformation, using `PIXEL_W` and `PIXEL_H` that are defined in "`do_not_edit.h`"
4. Rasterisation, which must follow the pseudocode in Ch2 Rendering including
    1. Calculate the barycentric coordinates
    2. Testing if a pixel is inside a triangle.
    3. Interpolating the vertex colours and depth values. Use `ShadeFragment()`.
    4. Merging using the `depth_buffer`.
    5. Writing colours to the `colour_buffer` using the provided `writeColToDisplayBuffer()`

When you run the program, the render should look like this



# Part B – Ray Tracer

In Part B you need to implement a Whitted Ray Tracer without refractions.

Only edit `'raytrace.h'` and submit only that single file to Moodle.

Open `Assessment1B.cpp` and look in `main()`. There are variables declared for performance analysis. The variable to store the path of the OBJ file to render is a global variable at the top of the file together with an eye position. The OBJ file is parsed using `obj_parse()` and the geometry is stored in a variable called `tris`.

Have a look at `obj_parse()` in "`do_not_edit.h`". A list of triangles is constructed from the data read using `tiny_obj_loader`. Special case code is there for "cornell2". In particular note that two of the "cornell2" triangles are reflective.

The method `raytrace()` is called from `main()`. Then the `pixelBuffer` is saved to a file called "`render.bmp`". Run the program and "`render.bmp`" should be generated, and you can open it in Microsoft paint, and should see a completely black image.

Your task is to implement `raytrace()` and all the necessary functions to raytrace the triangles. Open "`raytrace.h`" and you will see all the methods you need to implement.

Make sure

i) everything you implement is in "`raytrace.h`" since you can only submit that file,
ii) your raytracer renders the triangles stored in the vector declared as a global variable called `std::vector<triangle> tris` in "`do_not_edit.h`",
iii) you use the method `writeCol()` found in "`do_not_edit.h`" to write to the `pixelBuffer` which is saved to file at the end of `main()`.

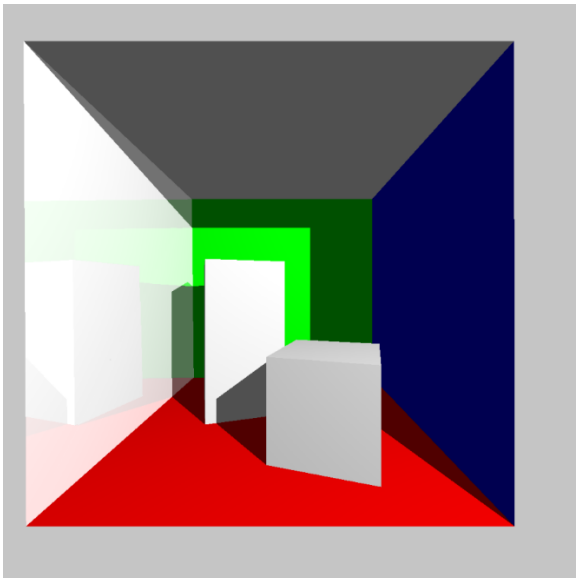
You will need to implement the pseudocode from Chapter 8 Ray Tracing.

You will notice at the top of "`do_not_edit.h`" that there is a `typedef closest_hit`, and in "`raytrace.h`" which is called for the closest intersecting triangle with a ray. There is a method `Shade()` for calculating a colour and `DoNothing()` to be used when testing shadow rays.

When calculating a ray direction use 90 degrees in radians for the `vfov`, and $R$ = (1, 0, 0), $U$ = (0, -1, 0), $F$ = (0, 0, −1).

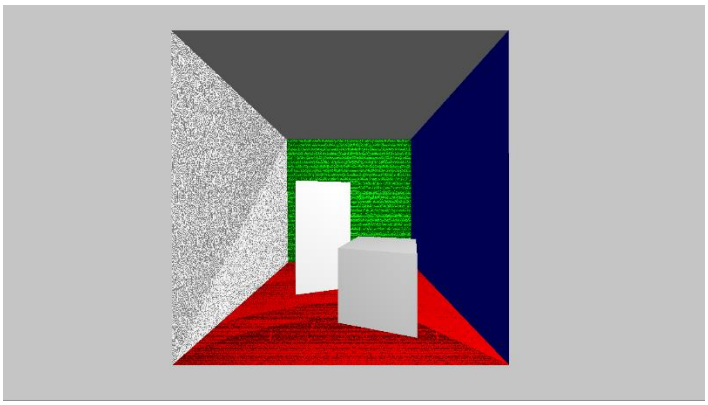
When implementing `Shade` use the colour of any vertex for the surface colour, ambient should be 0.1 and use the normal of any of the triangle vertices for the surface normal. You do not need to implement the refraction ray for the assessment.

The final implementation should produce a render that looks like this



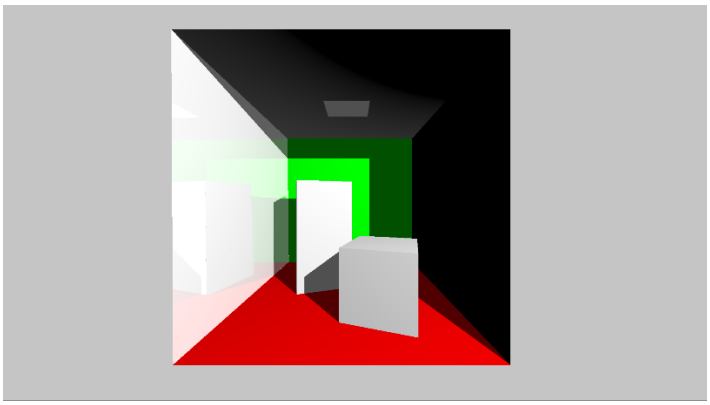
# Hints

If your raytraced render looks something like this



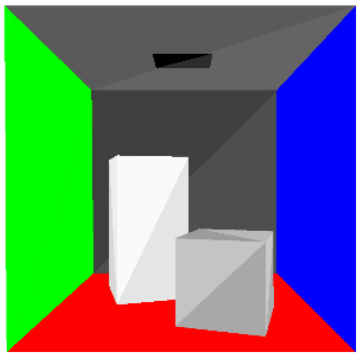
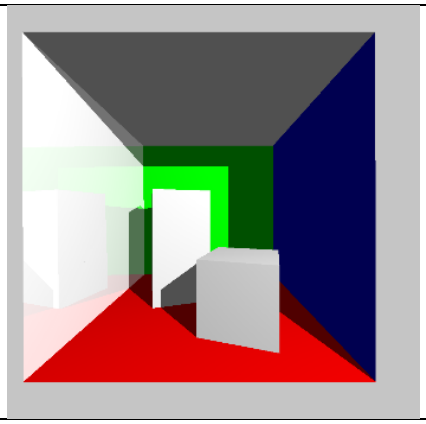
it probably means a secondary ray for shadows or reflections is intersecting with the same triangle it reflected off.

If your raytraced render looks something like this



this is probably because you are using diffuse values which are negative when the light is on the wrong side of geometry.

# Marks

| | | Marks | Time Limit |
|---|---|---|---|
| Rasterised “cornell2” |  | 15 | 60 seconds |
| Raytraced “cornell2” |  | 15 | 60 seconds |
| **TOTAL** | | **30** | |

You have been provided on Moodle with two samples called “rasterise.bmp”, and “raytrace.bmp” along with a zipped up program called `ImageCompare`. You can compile and run it in A32 or on your own computer. When you run it with your render and the sample render on Moodle, the program will provide statistics of the number of pixels in your render that match the sample render. The mark you receive will be proportional to the percentage of matching pixels in each render.

For example, using the command line on windows, run

```
ImageCompare.exe your_raster.bmp rasterise.bmp
```

The mark you get for the rasteriser will be proportional to the percentage of matching pixels.

You will then be penalised for the following:

1. If your program requires any changes to make it run.
2. If your program crashes.
3. If you use any API not provided on Moodle in the coursework .sln files.
4. If your program requires more than the times listed above to execute, excluding compilation time.
5. If your code visibly does not use the pseudocode in the lecture notes.